TASK 12 – RE-EVALUATE AND ADAPT THE DESIGN

A. Impact Analysis (2 Marks)

1. Changes in the Original Problem

The original program calculated electricity bills using fixed tariff rates and unit-based slabs. The new requirement uses smart meters that record energy consumption in watt-hours every 15 minutes, giving 35,040 readings per year. The tariff also changes according to the time of day, with a 20% higher rate between 6 PM and 10 PM.

2. Changes in Data Types

The program must handle watt-hour readings and yearly energy totals. A wider integer type, such as "long long", may be required for accumulated energy and billing values. The required data types should be selected after calculating the maximum possible totals.

3. Changes in Constants

Fixed tariff values defined using "#define" are no longer suitable because the tariff board wants to change rates without recompiling the program. The tariff values should instead be loaded from a configuration file when the program starts.

B. Evaluation of the Original Design (2 Marks)

The original design used named constants, integer paise, and fixed tariff slabs. These choices were suitable for a simple billing system because they provided accurate monetary calculations and made the code easy to understand.

However, fixed macros cannot conveniently support tariff changes at runtime. The original design also does not account for watt-hour readings or different rates during peak hours.

A 64-bit integer such as "long long" provides a larger range for yearly energy totals and accumulated charges. The maximum values must still be checked to prevent overflow.

C. Adaptation or Redesign (2 Marks)

The revised design will use the following changes:

1. Read smart-meter consumption in watt-hours every 15 minutes.

2. Use "long long" for accumulated energy and monetary calculations where required.

3. Load tariff rates and time periods from a configuration file at program startup.

4. Apply the normal rate during ordinary hours and a 20% higher rate between 6 PM and 10 PM.

5. Store monetary amounts in integer paise and apply a consistent rounding rule.

6. Validate meter readings and tariff configuration values before calculating bills.

This design allows tariff changes without recompiling the program and supports time-based billing.

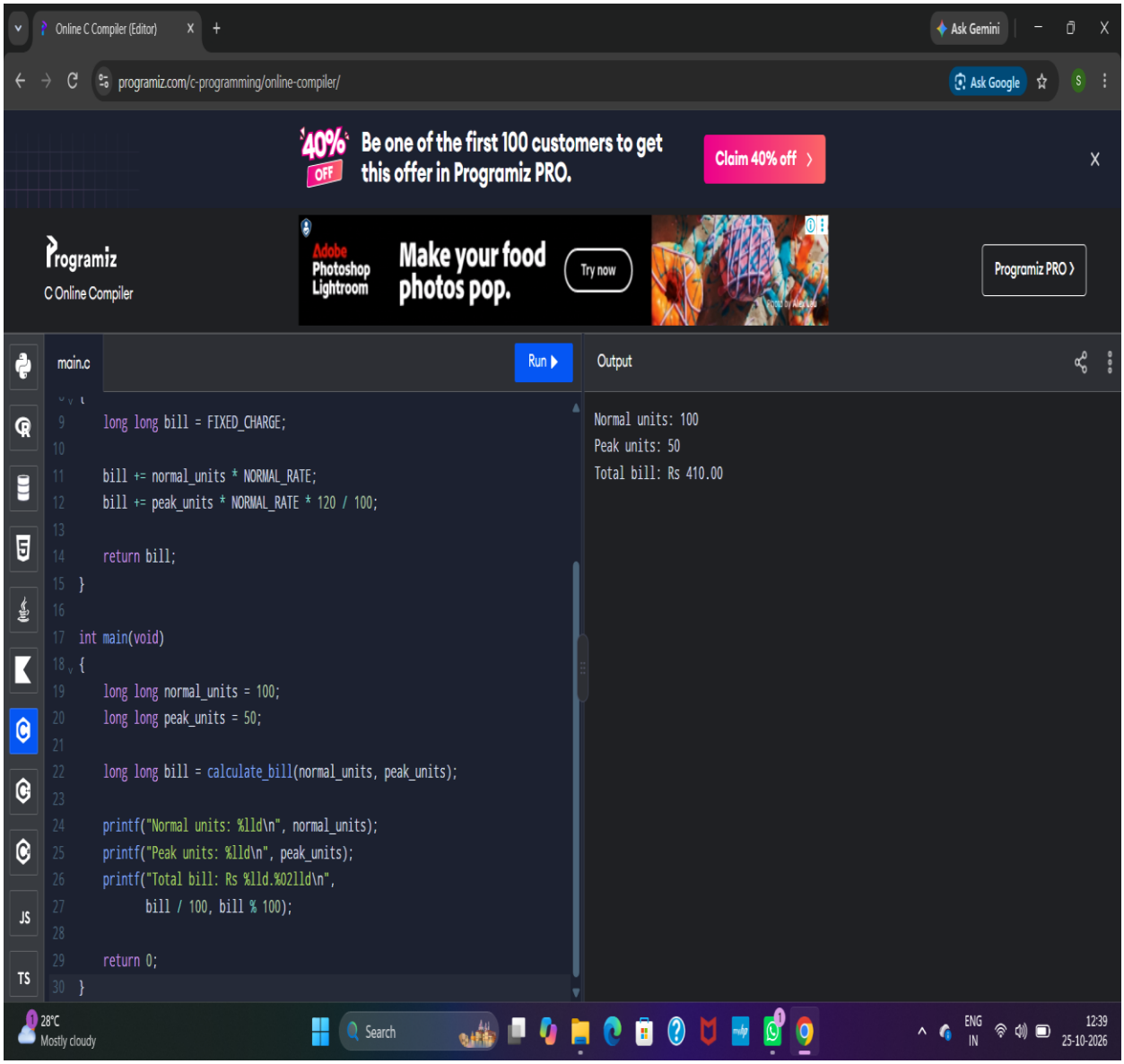
D. Evidence-Based Defence (2 Marks)

The revised design is more flexible than the original design because tariff rates can be changed through a configuration file. Using integer paise reduces floating-point rounding errors, while wider integer types allow larger totals to be stored.

The program should be tested with ordinary-hour readings, peak-hour readings, maximum yearly consumption, invalid configuration values, and boundary times such as 6 PM and 10 PM.

The final design should be accepted after the test results confirm correct billing, consistent rounding, and safe handling of large values.

SOURCE CODE


Online C Compiler (Editor)
Ask Gemini
programiz.com/c-programming/online-compiler/
Ask Google
40% OFF
Be one of the first 100 customers to get this offer in Programiz PRO.
Claim 40% off
Programiz
C Online Compiler
Adobe Photoshop Lightroom
Make your food photos pop.
Try now
Programiz PRO
main.c
Run
Output
long long bill = FIXED_CHARGE;
bill += normal_units * NORMAL_RATE;
bill += peak_units * NORMAL_RATE * 120 / 100;
return bill;
}
int main(void)
{
long long normal_units = 100;
long long peak_units = 50;
long long bill = calculate_bill(normal_units, peak_units);
printf("Normal units: %lld\n", normal_units);
printf("Peak units: %lld\n", peak_units);
printf("Total bill: Rs %lld.%02lld\n",
bill / 100, bill % 100);
return 0;
}
Normal units: 100
Peak units: 50
Total bill: Rs 410.00
28°C
Mostly cloudy
Search
ENG
IN
12:39
25-10-2026